

A complete Newton solver using Eigen

Paolo Joseph Baioni

April 24, 2026

Newton solver

This example (inspired by `Examples/src/NewtonSolver`) presents a set of tools implementing Newton methods for non-linear systems, using the Eigen library.

The code structure is the following:

- ▶ `NewtonTraits` defines the common types used by the solver classes, ensuring a uniform interface.
- ▶ `JacobianBase` is an abstract base class representing the action of a Jacobian (or approximate Jacobian).
- ▶ `FullJacobian` uses a user-provided Jacobian matrix.
- ▶ `DiscreteJacobian` approximates the Jacobian by finite differences.
- ▶ `JacobianFactory` returns a concrete object in the `JacobianBase` family.
- ▶ `Newton` applies the Newton iteration given a non-linear system and a Jacobian object.
- ▶ `NewtonOptions` and `NewtonResult` collect input parameters and output data.

Exercise

Consider the problem

$$\mathbf{f}(x, y) = \begin{bmatrix} (x - 1)^2 + 0.1(y - 5)^2 \\ 1.5 - x - 0.1y \end{bmatrix} = \mathbf{0}.$$

Starting from the provided solution sketch:

1. Implement the `NewtonTraits` class, defining the common types used by the solver.
2. Implement the `FullJacobian` class (derived from `JacobianBase`), which, given the full Jacobian matrix, solves the linear system by direct LU factorization.
3. Implement the `DiscreteJacobian` class (derived from `JacobianBase`), which approximates the Jacobian by finite differences and solves the resulting linear system by direct LU factorization.
4. Implement a `JacobianFactory` function returning an instance of `FullJacobian` or `DiscreteJacobian`, depending on the selected Jacobian type.
5. Solve the problem above using both the full and the discrete approach.
6. Extra: generalize the problem type (scalar, vector, ...) with templates, while keeping runtime Jacobian selection.